

Groma — Build Specification

Implementation guidance for Claude Code. Every section is written to be quoted directly into a prompt.

Conceptual background is not repeated here. Where a concept needs explaining, the reference is

`docs/explained.md §n`.

Companion documents:

- `docs/design.md` — engineering design
 - `docs/explained.md` — the same design from first principles
 - `CLAUDE.md` — repo-root conventions, read at the start of every session
-

Contents

1. [How to use this document](#)
2. [Delivery scope and order](#)
3. [Stack and repository layout](#)
4. [Contracts](#)
5. [Database schema](#)
6. [Coverage kernel](#)
7. [API surface](#)
8. [Capture ingest and quality gating](#)
9. [Reconstruction adapter](#)
10. [Georeferencing and accuracy reporting](#)
11. [Artefact processing and tiling](#)
12. [Structure extraction](#)
13. [Semantic lift from source imagery](#)
14. [Measurement and uncertainty](#)
15. [Placement optimisation and reporting](#)
16. [Synthetic ground truth](#)
17. [Milestones](#)
18. [Test strategy](#)
19. [Deployment](#)
20. [Session guidance](#)

1. How to use this document

- One milestone (§17) per Claude Code session. One sub-task per prompt.
 - Quote the relevant specification section into the prompt rather than relying on the file being read.
 - Write tests in the same prompt as the implementation. This system produces arrays and percentages that look plausible whether or not they are correct; an implementation-first workflow will not catch that.
 - Run `make test` at the start and end of every session.
 - Where this document gives a numeric expected value, it is a test assertion, not an illustration.
-

2. Delivery scope and order

Everything in §4–§15 ships. The system is not complete without the photogrammetry pipeline; a coverage planner that cannot ingest a drone survey is a CAD toy.

The build order below is driven by **testability**, not by scope reduction:

1. The coverage kernel comes first because it is the component with no substitute and the one whose bugs are hardest to see.
2. The synthetic ground truth (§16) is built early because it makes the photogrammetry stages testable.
`site_alpha` is a hand-authored site; from it we generate a synthetic point cloud and a set of rendered drone images with exactly known camera poses. Structure extraction is then validated by asking whether it recovers the primitives we authored, to within 10 cm. Reconstruction is validated by asking whether it recovers the poses we rendered from.
3. Real-imagery reconstruction lands as soon as the job system exists to carry it.

No milestone is optional. If schedule pressure appears, the response is to reduce `--pc-quality`, not to drop a stage.

3. Stack and repository layout

3.1 Stack

Layer	Choice
Backend	Python 3.12, FastAPI, Pydantic v2
Numerics	NumPy, SciPy
Geometry	Shapely 2.x, pyproj
Point clouds	laspy, PDAL (CLI), Open3D
Rasters	rasterio, GDAL
Imagery	Pillow, OpenCV, exiftool (subprocess), ffmpeg (subprocess)

Reconstruction	NodeODM (HTTP), COLMAP (subprocess), pycolmap
Segmentation	ultralytics YOLO-seg, segment-anything-2 (M11, GPU)
Database	PostgreSQL 16 + PostGIS 3.4
Migrations	Alembic
Jobs	arq + Redis
Tiling	py3dtiles, Entwine, Draco
Frontend	React 18, TypeScript, Vite, three.js, react-three-fiber
Testing	pytest, hypothesis, testcontainers, Playwright
Serving	nginx single-port, supervisord

3.2 Layout

```

groma/
├─ CLAUDE.md
├─ pyproject.toml           # uv workspace
├─ docker-compose.yml      # postgres+postgis, redis, nodeodm
├─ Makefile
├─ docs/                   # design.md, explained.md, build-spec.md, runbook.md
├─
├─ packages/
│  ├─ contracts/groma_contracts/
│  │  │  └─ version.py geometry.py site.py survey.py imagery.py
│  │  └─ structure.py camera.py scenario.py coverage.py
│  │  └─ measurement.py jobs.py
│  │
│  └─ geo/groma_geo/
│  │  └─ crs.py origin.py heights.py optics.py raster.py
│  │
│  └─ coverage/groma_coverage/
│  │  └─ types.py occluders.py terrain.py kernel.py
│  │  └─ stats.py optimise.py reference.py
│  │
│  └─ capture/groma_capture/
│  │  └─ exif.py sensors.py quality.py overlap.py video.py gcp.py
│  │
│  └─ recon/groma_recon/
│  │  └─ base.py fixture.py odm.py colmap.py
│  │  └─ artefacts.py validate.py scale_check.py
│  │
│  └─ tiles/groma_tiles/
│  │  └─ pointcloud.py mesh.py terrain.py ortho.py
│  │
│  └─ segment/groma_segment/
│  │  └─ ground.py cluster.py fit.py descriptors.py
│  │  └─ classify.py lift.py mounts.py
│  │
├─ apps/
│  └─ api/groma_api/       # main, deps, db/, routers/, ws.py
│  └─ worker/groma_worker/ # tasks/: ingest, reconstruct, tile, extract, coverage, optimise

```

```

|   |─ cli/groma_cli/
|   └─ web/src/                # api/, scene/, kernel/, panels/, review/
|
└─ scripts/
|   |─ synthesise_site.py      # site_alpha → point cloud + DSM/DTM
|   └─ render_survey.py       # site_alpha → drone images with known poses
|
└─ migrations/
└─ deploy/autodl/             # bootstrap.sh, nginx.conf, supervisord.conf
└─ fixtures/
|   |─ sites/site_alpha.json
|   |─ recon/site_alpha/      # generated by scripts/
|   └─ sensors/sensors.yaml   # sensor dimension lookup table
└─ tests/
    |─ unit/  integration/  golden/  e2e/

```

Dependency rule, enforced by `tests/unit/test_import_graph.py`: `apps/*` → `packages/*` → `packages/contracts`. Never the reverse.

4. Contracts

Define once in Pydantic; generate TypeScript. Never hand-write a duplicate type.

4.1 Geometry

```

class Vec3(BaseModel):
    x: float; y: float; z: float          # local ENU metres, Y up

class BoxPrim(BaseModel):
    kind: Literal["box"] = "box"
    cx: float; cy: float; cz: float
    hx: float; hy: float; hz: float
    yaw: float = 0.0                     # radians about Y

class CylinderPrim(BaseModel):
    kind: Literal["cylinder"] = "cylinder"
    cx: float; cz: float; r: float
    y0: float; y1: float

class ExtrudedPolyline(BaseModel):      # fence runs
    kind: Literal["polyline"] = "polyline"
    points: list[tuple[float, float]]    # plan view, local ENU
    y0: float; y1: float
    thickness: float

Primitive = BoxPrim | CylinderPrim | ExtrudedPolyline  # discriminated on `kind`

```

4.2 Imagery and capture

```

class SensorSpec(BaseModel):
    make: str; model: str
    sensor_w_mm: float; sensor_h_mm: float
    res_x: int; res_y: int

```

```

class SourceImage(BaseModel):
    id: str
    survey_id: str
    filename: str
    uri: str
    sha256: str
    width: int; height: int
    focal_mm: float | None
    sensor: SensorSpec | None
    captured_at: datetime | None
    gps: Vec3 | None # storage CRS, from EXIF
    gps_accuracy_m: float | None
    rtk_fixed: bool = False
    gimbal_pitch_deg: float | None # -90 = nadir
    gimbal_yaw_deg: float | None
    sharpness: float | None # variance of Laplacian
    clipped_fraction: float | None
    state: Literal["accepted", "rejected_blur", "rejected_exposure", "rejected_manual"]
    source: Literal["still", "video_frame"]
    video_frame_index: int | None

class CaptureQA(BaseModel):
    image_count: int
    accepted_count: int
    rejected: dict[str, int]
    sharpness_p10: float
    estimated_gsd_m: float | None
    estimated_front_overlap: float | None # 0..1
    estimated_side_overlap: float | None
    nadir_count: int
    oblique_count: int
    rtk_fraction: float
    warnings: list[str]
    blocking: list[str] # must be acknowledged before reconstruct

```

4.3 Survey and georeferencing

```

class GeorefMethod(StrEnum):
    RTK = "rtk"; PPK = "ppk"; GCP = "gcp"; SCALE_BAR = "scale_bar"; NONE = "none"

class GcpObservation(BaseModel):
    gcp_id: str
    image_id: str
    px: float; py: float

class Gcp(BaseModel):
    id: str
    survey_id: str
    label: str
    easting: float; northing: float; height: float
    role: Literal["control", "check"] # check points are held out
    observations: list[GcpObservation]

class AccuracyReport(BaseModel):
    reproj_rmse_px: float | None
    gcp_rmse_h_m: float | None # control points — always optimistic

```

```

gcp_rmse_v_m: float | None
check_rmse_h_m: float | None          # the honest number
check_rmse_v_m: float | None
check_point_count: int
gsd_m: float | None
scale_error_pct: float | None         # from $10.4
registered_images: int
total_images: int

```

4.4 Structures, cameras, scenarios, coverage

```

class StructureClass(StrEnum):
    POLE = "pole"; FENCE = "fence"; BUILDING = "building"; STAND = "stand"
    GOAL = "goal"; VEGETATION = "vegetation"; GROUND = "ground"; OTHER = "other"

class ReviewState(StrEnum):
    PENDING = "pending"; ACCEPTED = "accepted"
    REJECTED = "rejected"; SEASONAL = "seasonal"

class RejectReason(StrEnum):
    NOISE = "noise"; TRANSIENT = "transient"; DUPLICATE = "duplicate"

class Structure(BaseModel):
    id: str; survey_id: str
    cls: StructureClass
    name: str
    confidence: float = Field(ge=0, le=1)
    state: ReviewState = ReviewState.PENDING
    reject_reason: RejectReason | None = None
    primitive: Primitive
    porosity: float = Field(default=0.0, ge=0, le=1)    # 0 = solid
    mountable: bool = False
    fit_rmse_m: float | None
    point_count: int | None
    origin: Literal["extracted", "manual", "adjusted"]
    reviewed_by: str | None; reviewed_at: datetime | None

class CameraSpec(BaseModel):
    id: str; name: str
    position: Vec3          # lens position, bracket already applied
    pan_deg: float          # 0 = -Z, increasing clockwise from above
    tilt_deg: float         # positive = downward
    roll_deg: float = 0.0
    sensor_w_mm: float; sensor_h_mm: float
    focal_mm: float
    res_x: int; res_y: int
    near_m: float = 1.0; far_m: float = 200.0
    mount_structure_id: str | None = None    # excluded from this camera's occluders
    bracket_offset_m: float = 0.0
    enabled: bool = True

```

Pan/tilt convention, stated once:

```

forward = ( sin(pan)·cos(tilt), -sin(tilt), -cos(pan)·cos(tilt) )
right   = normalise( forward × (0,1,0) )
up      = right × forward

```

```

class DoriTier(StrEnum):
    IDENTIFY = "identify"; RECOGNISE = "recognise"
    OBSERVE = "observe"; DETECT = "detect"

DORI_PX_PER_M = {IDENTIFY: 250.0, RECOGNISE: 125.0, OBSERVE: 62.0, DETECT: 25.0}

class CoverageRequest(BaseModel):
    scenario_id: str
    eval_height_m: float = 1.6          # above local terrain, not above datum
    grid_spacing_m: float = 0.5
    include_tents: bool = True
    include_seasonal: bool = True
    foreshorten: bool = True
    use_terrain: bool = True
    method: Literal["raycast", "shadowmap"] = "raycast"

class CoverageStats(BaseModel):
    kernel_version: str
    cells: int; cell_area_m2: float; area_m2: float
    tier_area_m2: dict[DoriTier, float]      # cumulative
    below_detect_m2: float
    blind_m2: float
    redundant_2plus_m2: float
    per_camera_unique_m2: dict[str, float]   # area only this camera covers
    mean_ppm: float

```

5. Database schema

Alembic 0001_initial . PostGIS geometry columns use the site's SRID.

```

CREATE EXTENSION IF NOT EXISTS postgis;

CREATE TABLE site (
    id          uuid PRIMARY KEY,
    name        text NOT NULL,
    srid        integer NOT NULL,
    origin_x    double precision NOT NULL,
    origin_y    double precision NOT NULL,
    origin_z    double precision NOT NULL,
    height_datum text NOT NULL,      -- orthometric_mpd | ellipsoidal | local
    created_at  timestampz NOT NULL DEFAULT now()
);

CREATE TYPE georef_method AS ENUM ('rtk', 'ppk', 'gcp', 'scale_bar', 'none');
CREATE TYPE survey_status AS ENUM
    ('draft', 'ingesting', 'qa_review', 'queued', 'reconstructing',
     'processing', 'extracting', 'complete', 'failed');

CREATE TABLE survey (
    id          uuid PRIMARY KEY,
    site_id     uuid NOT NULL REFERENCES site(id) ON DELETE CASCADE,
    name        text NOT NULL,
    flown_at    date,

```

```

platform      text,
georef        georef_method NOT NULL DEFAULT 'none',
status        survey_status NOT NULL DEFAULT 'draft',
engine        text,                -- odm-3.5.4 | colmap-3.11 | fixture | import
capture_qa    jsonb,
accuracy      jsonb,                -- AccuracyReport
immutable     boolean NOT NULL DEFAULT false,
superseded_by uuid REFERENCES survey(id),
created_at    timestampz NOT NULL DEFAULT now()
);
CREATE INDEX ON survey (site_id, flown_at DESC);

CREATE TABLE source_image (
  id          uuid PRIMARY KEY,
  survey_id   uuid NOT NULL REFERENCES survey(id) ON DELETE CASCADE,
  filename    text NOT NULL,
  uri         text NOT NULL,
  sha256      char(64) NOT NULL,
  width       integer NOT NULL, height integer NOT NULL,
  focal_mm    real, sensor_w_mm real, sensor_h_mm real,
  captured_at timestampz,
  gps         geometry(PointZ, 0),
  gps_accuracy_m real,
  rtk_fixed   boolean NOT NULL DEFAULT false,
  gimbal_pitch_deg real, gimbal_yaw_deg real,
  sharpness   real, clipped_fraction real,
  state       text NOT NULL DEFAULT 'accepted',
  source      text NOT NULL DEFAULT 'still',
  video_frame_index integer,
  pose        jsonb,                -- filled after reconstruction: R, t, intrinsics
  UNIQUE (survey_id, filename)
);
CREATE INDEX ON source_image (survey_id) WHERE state = 'accepted';
CREATE INDEX ON source_image USING gist (gps);

CREATE TABLE gcp (
  id          uuid PRIMARY KEY,
  survey_id   uuid NOT NULL REFERENCES survey(id) ON DELETE CASCADE,
  label       text NOT NULL,
  position    geometry(PointZ, 0) NOT NULL,
  role        text NOT NULL DEFAULT 'control',  -- control | check
  residual_h_m real, residual_v_m real          -- filled after reconstruction
);

CREATE TABLE gcp_observation (
  id          uuid PRIMARY KEY,
  gcp_id      uuid NOT NULL REFERENCES gcp(id) ON DELETE CASCADE,
  image_id    uuid NOT NULL REFERENCES source_image(id) ON DELETE CASCADE,
  px          real NOT NULL, py real NOT NULL,
  UNIQUE (gcp_id, image_id)
);

CREATE TABLE artefact (
  id          uuid PRIMARY KEY,
  survey_id   uuid NOT NULL REFERENCES survey(id) ON DELETE CASCADE,
  kind        text NOT NULL,        -- pointcloud|dsm|dtm|ortho|mesh|poses|tiles_pc|
                                     -- tiles_mesh|tiles_ortho|terrain_grid|report
  uri         text NOT NULL,

```

```

        bytes      bigint, sha256 char(64) NOT NULL,
        meta       jsonb,          -- bounds, srid, resolution, point count
        created_at timestamptz NOT NULL DEFAULT now(),
        UNIQUE (survey_id, kind)
    );

CREATE TYPE review_state AS ENUM ('pending','accepted','rejected','seasonal');

CREATE TABLE structure (
    id          uuid PRIMARY KEY,
    survey_id   uuid NOT NULL REFERENCES survey(id) ON DELETE CASCADE,
    cls         text NOT NULL,
    name        text NOT NULL,
    confidence   real NOT NULL,
    state       review_state NOT NULL DEFAULT 'pending',
    reject_reason text,
    primitive   jsonb NOT NULL,
    porosity    real NOT NULL DEFAULT 0,
    mountable   boolean NOT NULL DEFAULT false,
    fit_rmse_m  real,
    point_count integer,
    origin      text NOT NULL DEFAULT 'extracted',
    footprint   geometry(PolygonZ, 0),
    evidence    jsonb,          -- [{image_id, bbox}] best 3 source views
    reviewed_by text, reviewed_at timestamptz
);

CREATE INDEX ON structure USING gist (footprint);
CREATE INDEX ON structure (survey_id) WHERE state = 'accepted';

CREATE TABLE mount_point (
    id          uuid PRIMARY KEY,
    structure_id uuid NOT NULL REFERENCES structure(id) ON DELETE CASCADE,
    position     geometry(PointZ, 0) NOT NULL,
    max_load_kg  real, label text
);

CREATE TABLE scenario (
    id          uuid PRIMARY KEY,
    site_id     uuid NOT NULL REFERENCES site(id) ON DELETE CASCADE,
    base_survey_id uuid NOT NULL REFERENCES survey(id),
    name        text NOT NULL,
    include_seasonal boolean NOT NULL DEFAULT true,
    created_at   timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE camera (
    id          uuid PRIMARY KEY,
    scenario_id  uuid NOT NULL REFERENCES scenario(id) ON DELETE CASCADE,
    mount_point_id  uuid REFERENCES mount_point(id),
    mount_structure_id uuid REFERENCES structure(id),
    name         text NOT NULL,
    position      geometry(PointZ, 0) NOT NULL,
    pan_deg real NOT NULL, tilt_deg real NOT NULL, roll_deg real NOT NULL DEFAULT 0,
    bracket_offset_m  real NOT NULL DEFAULT 0,
    sensor_w_mm real NOT NULL, sensor_h_mm real NOT NULL, focal_mm real NOT NULL,
    res_x integer NOT NULL, res_y integer NOT NULL,
    near_m real NOT NULL DEFAULT 1, far_m real NOT NULL DEFAULT 200,
    model_name text, enabled boolean NOT NULL DEFAULT true

```

```

);

CREATE TABLE tent (
    id            uuid PRIMARY KEY,
    scenario_id   uuid NOT NULL REFERENCES scenario(id) ON DELETE CASCADE,
    name          text NOT NULL,
    footprint     geometry(Polygon, 0) NOT NULL,
    height_m      real NOT NULL,
    yaw_deg       real NOT NULL DEFAULT 0
);

CREATE TABLE coverage_run (
    id            uuid PRIMARY KEY,
    scenario_id   uuid NOT NULL REFERENCES scenario(id) ON DELETE CASCADE,
    eval_height_m real NOT NULL, grid_spacing_m real NOT NULL,
    include_tents boolean NOT NULL, foreshorten boolean NOT NULL,
    use_terrain   boolean NOT NULL, method text NOT NULL,
    kernel_version text NOT NULL,
    stats         jsonb NOT NULL,
    grid_uri      text,
    blind_polygons geometry(MultiPolygon, 0),
    computed_at   timestampz NOT NULL DEFAULT now(),
    duration_ms   integer
);

CREATE TABLE measurement (
    id            uuid PRIMARY KEY,
    site_id       uuid NOT NULL REFERENCES site(id) ON DELETE CASCADE,
    survey_id     uuid NOT NULL REFERENCES survey(id),
    kind          text NOT NULL,
    geom          geometry(GeometryZ, 0) NOT NULL,
    value         double precision NOT NULL,
    uncertainty    double precision NOT NULL,      -- deliberately NOT NULL
    unit          text NOT NULL,
    snap_mode     text NOT NULL,
    created_by    text, created_at timestampz NOT NULL DEFAULT now()
);

CREATE TABLE job (
    id            uuid PRIMARY KEY,
    kind          text NOT NULL,
    ref_id        uuid,
    status        text NOT NULL,
    progress      real NOT NULL DEFAULT 0,
    stage         text, message text, error text,
    external_id   text,                      -- NodeODM task uuid, for re-attach
    started_at    timestampz, finished_at timestampz
);

```

Constraints to enforce in the API layer and test:

- POST /measurements returns **409** when survey.georef = 'none'.
- POST /surveys/{id}/reconstruct returns **409** when capture_ga.blocking is non-empty and unacknowledged.
- A survey with immutable = true rejects all mutations except superseded_by.

6. Coverage kernel

Pure package. NumPy in, NumPy out. No I/O, no database, no framework, no logging of user data. Compiles to WASM.

6.1 Types

```
KERNEL_VERSION = "1.0.0"    # bump on ANY behavioural change

@dataclass(frozen=True)
class Occluder:
    id: str
    prim: BoxPrim | CylinderPrim | ExtrudedPolyline
    owner_id: str | None      # structure or tent id; used for mount exclusion
    porosity: float = 0.0     # 0 = solid, 1 = fully transparent

@dataclass(frozen=True)
class Terrain:
    """DTM as a heightfield in local ENU. Occludes, and defines eval height."""
    x_min: float; z_min: float
    spacing: float
    heights: np.ndarray        # float32 (nz, nx), Y in local ENU
    def height_at(self, x, z) -> np.ndarray: ...    # bilinear, vectorised

@dataclass(frozen=True)
class Grid:
    x_min: float; x_max: float; z_min: float; z_max: float; spacing: float
    @property
    def nx(self) -> int: ...
    @property
    def nz(self) -> int: ...
    def centres(self) -> tuple[np.ndarray, np.ndarray]:
        """(X, Z) of shape (nz, nx). Row 0 is z_min, column 0 is x_min.
        Never change this ordering; the heatmap texture depends on it."""

@dataclass
class CoverageResult:
    ppm: np.ndarray            # float32 (nz, nx), best px/m; 0 = no sightline
    count: np.ndarray           # uint8   (nz, nx)
    best_camera: np.ndarray     # int16   (nz, nx), -1 if none
    eval_y: np.ndarray          # float32 (nz, nx), absolute height evaluated at
    grid: Grid
    kernel_version: str
```

6.2 Optics — packages/geo/groma_geo/optics.py

```
def hfov_rad(sensor_w_mm, focal_mm)    -> 2*atan(sensor_w_mm / (2*focal_mm))
def vfov_rad(sensor_h_mm, focal_mm)    -> 2*atan(sensor_h_mm / (2*focal_mm))
def f_px(focal_mm, res_y, sensor_h_mm) -> focal_mm * res_y / sensor_h_mm
def gsd_m(sensor_w_mm, altitude_m, focal_mm, res_x)
    -> sensor_w_mm*altitude_m/(focal_mm*res_x)
def dori_range_m(f_px_value, px_per_m) -> f_px_value / px_per_m
def footprint_m(altitude_m, fov_rad)    -> 2*altitude_m*tan(fov_rad/2)
```

6.3 Entry point

```
def compute_coverage(
    cameras: Sequence[CameraSpec],
    occluders: Sequence[Occluder],
    grid: Grid,
    terrain: Terrain | None = None,
    eval_height_m: float = 1.6,          # ABOVE LOCAL TERRAIN
    foreshorten: bool = True,
) -> CoverageResult: ...

def summarise(result, tiers=DORI_PX_PER_M) -> CoverageStats: ...
def blind_polygons(result, min_area_m2=4.0) -> list[Polygon]: ... # marching squares
def compare(a: CoverageResult, b: CoverageResult) -> CoverageDelta: ...
```

`eval_height_m` is **above the local terrain surface**, not above the datum. Where `terrain` is `None`, treat terrain as the plane $y = 0$. `CoverageResult.eval_y` records the absolute height actually used per cell, so reports can show it.

6.4 Algorithm, per camera

1. $V = P - \text{camera.position}$, `shape (nz, nx, 3)`. $d2 = (V**2).sum(-1)$.
2. Range gate: $\text{mask} = (d2 \geq \text{near}^2) \ \& \ (d2 \leq \text{far}^2)$.
3. Project into the camera basis: $z_c = V @ f$, $x_c = V @ r$, $y_c = V @ u$. $\text{mask} \ \&= \ (z_c > 0) \ \& \ (|x_c| \leq z_c \cdot \tan_h) \ \& \ (|y_c| \leq z_c \cdot \tan_v)$.
4. Reduce to surviving candidate indices (typically 10–40% of cells).
5. Occlusion, per occluder, vectorised over candidates:
 - Broad phase: vertical extent overlap, then plan-view distance from the occluder's bounding circle to the segment.
 - Exact: slab test (box, and polyline segments as rotated boxes), quadratic (cylinder).
 - Solid occluder $\rightarrow$ mark blocked. Porous occluder $\rightarrow$ accumulate transmission $\prod (1 - \text{porosity}_i)$ rather than blocking.
6. Terrain occlusion: DDA ray-march the heightfield along the segment's plan-view projection, comparing ray height to interpolated terrain height at each raster step. Blocked if the ray dips below terrain anywhere in $(\varepsilon, 1-\varepsilon)$.
7. $\text{ppm} = f_{\text{px}} / d$; if `foreshorten`, $\times \sqrt{1 - (V.y/d)^2}$; $\times \text{transmission}$.
8. Accumulate with `np.maximum`, increment `count`, record `best_camera`.

Exclusions. If `cam.mount_structure_id` is set, filter that owner from the camera's occluder list. Start rays at $t = \varepsilon$ corresponding to 2 cm.

Performance target. 173,184 cells (0.25 m over 132×82 m) $\times$ 6 cameras $\times$ 30 occluders + terrain, under 800 ms.

6.5 reference.py

A second implementation: plain Python loops, one cell at a time, no vectorisation, no broad phase, no cleverness. Fifty lines, obviously correct by inspection. Used to validate the fast kernel and to debug wrong maps.

6.6 Required tests, with closed-form expected values

tests/unit/test_kernel_analytic.py.

	Test	Expected
T1	<code>f_px</code> identity	<code>f_px(f, ry, sh) == ry / (2 * tan(vfov_rad(sh, f) / 2))</code> to 1e-9
T2	Nadir footprint, camera at height <code>h</code> , tilt 90°, flat terrain, eval 0	<code>area = 2h * tan(HFOV/2) * 2h * tan(VFOV/2)</code> ; peak ppm = <code>f_px/h</code> within 0.5%
T3	Wall shadow. Camera <code>(0, h, 0)</code> looking +X, tilt 0; wall height <code>w</code> at distance <code>a</code> ; eval height <code>e</code>	shadow ends at <code>x = a * (h - e) / (h - w)</code> . With <code>h=10, w=3, a=20, e=1.6</code> → exactly 24.0 m, ±1 cell
T4	Grid invariance at 1.0 / 0.5 / 0.25 m	every tier percentage within 0.5 pp
T5	Fast kernel vs <code>reference.py</code> , 40×40 grid, 3 cameras, 5 occluders, terrain on	<code>ppm</code> within 1e-4; <code>count</code> identical
T6	Monotonicity	adding an occluder never raises ppm; adding a camera never lowers ppm or count; disabling == removing
T7	Foreshortening bound	<code>ppm(True) ≤ ppm(False)</code> everywhere; equal iff <code>v.y == 0</code>
T8	Self-occlusion	camera on a cylinder axis: with <code>mount_structure_id</code> set, that cylinder has no effect; without it, >40% of the frustum is blocked
T9	Pan cardinals	<code>pan=0</code> covers -Z; <code>pan=90</code> covers +X; <code>pan=180</code> covers +Z; <code>pan=-90</code> covers -X
T10	Terrain occlusion	camera behind a ridge of height <code>w</code> at distance <code>a</code> : shadow matches the T3 formula
T11	Terrain eval height	on a 5% slope, <code>eval_y</code> equals <code>terrain + 1.6</code> everywhere within 1 mm
T12	Porosity	occluder with <code>porosity=0.5</code> halves ppm behind it, does not zero it
T13	Golden fixture	<code>site_alpha</code> , table below, ±0.3 pp
T14	Import graph	<code>no packages/* imports apps/*</code>

T13 reference values. `site_alpha`, 0.5 m grid, 1.6 m eval, foreshortening on, flat terrain, no tents, all structures accepted, four corner-mast cameras (8 mm lens, 1/2.8" sensor 5.37 × 4.04 mm, 3840 × 2160, 14 m mount, 18° tilt, aimed at centre):

Metric	Expected
Recognise or better	5.2%
Observe or better	38.6%

Detect or better	91.7%
Blind	8.3%
Seen by 2+ cameras	51.3%

With the 3 × 4 tent grid (8 × 8 × 3.2 m at 20 m / 14 m spacing, centred on the pitch): blind rises to **21.2%**, roughly **1,400 m²** newly without sightline.

Never widen the tolerance to make this pass. Investigate the divergence.

7. API surface

Sites

```
POST  /api/sites                → Site
GET   /api/sites/{id}          → Site
GET   /api/sites/{id}/surveys  → [Survey]
```

Capture

```
POST  /api/sites/{id}/surveys  → Survey (draft)
POST  /api/surveys/{id}/images:init  → upload session, presigned parts
POST  /api/surveys/{id}/images:complete  → Job (ingest + QA)
POST  /api/surveys/{id}/video  → Job (keyframe extraction + QA)
GET   /api/surveys/{id}/qa     → CaptureQA
POST  /api/surveys/{id}/qa:acknowledge  → Survey (clears blocking)
GET   /api/surveys/{id}/images  → [SourceImage] (keyset paginated)
PATCH /api/images/{id}        → SourceImage (manual reject/restore)
```

Ground control

```
POST  /api/surveys/{id}/gcps    → Gcp
POST  /api/gcps/{id}/observations  → GcpObservation
GET   /api/surveys/{id}/gcps/suggest/{gcp_id} → [image_id] likely to contain it
DELETE /api/gcp-observations/{id}
```

Reconstruction

```
POST  /api/surveys/{id}/reconstruct  → Job
POST  /api/surveys/{id}/import        → Job (external LAZ/GeoTIFF/OBJ)
POST  /api/surveys/{id}/cancel        → Job
GET   /api/surveys/{id}/accuracy      → AccuracyReport
GET   /api/surveys/{id}/artefacts     → [Artefact]
POST  /api/surveys/{id}/supersede     → Survey (new draft, copies GCPs)
```

Structure extraction and review

```
POST  /api/surveys/{id}/extract  → Job
GET   /api/surveys/{id}/structures  → [Structure] (keyset paginated)
GET   /api/structures/{id}/evidence  → thumbnail crops + source image refs
PATCH /api/structures/{id}        → Structure (state/class/primitive)
POST  /api/structures/{id}/refit    → Structure (after reclassify)
POST  /api/structures:bulk-review   → [Structure]
POST  /api/surveys/{id}/structures  → Structure (manual, origin='manual')
GET   /api/surveys/{id}/mount-points → [MountPoint]
```

Scenarios

```
POST  /api/sites/{id}/scenarios  → Scenario
POST  /api/scenarios/{id}/clone  → Scenario
```

```
POST    /api/scenarios/{id}/cameras          → CameraSpec
PATCH  /api/cameras/{id} · DELETE /api/cameras/{id}
POST    /api/scenarios/{id}/tents          → Tent
PATCH  /api/tents/{id} · DELETE /api/tents/{id}

# Coverage
POST    /api/scenarios/{id}/coverage        → CoverageRun (sync if ≥0.5 m, else Job)
GET     /api/coverage-runs/{id}             → CoverageRun + CoverageStats
GET     /api/coverage-runs/{id}/grid.npz    → raw arrays
GET     /api/coverage-runs/{id}/grid.png    → DORI-coloured heatmap
POST    /api/coverage/compare               → CoverageDelta {run_a, run_b}
POST    /api/scenarios/{id}/optimise       → Job
GET     /api/coverage-runs/{id}/report.pdf → deliverable

# Measurement
POST    /api/measurements                  → Measurement (409 if georef='none')
GET     /api/sites/{id}/measurements       → [Measurement]

# Jobs
GET     /api/jobs/{id} · POST /api/jobs/{id}/cancel
WS      /ws/jobs/{id}                     ← JobProgress stream
```

Roles: viewer / surveyor / admin . Structure review and GCP marking require `surveyor` — the audit trail is worthless if anonymous. Keyset pagination on `/images` and `/structures`; both can run to thousands of rows.

8. Capture ingest and quality gating

`packages/capture/` . Runs as a job before reconstruction is permitted.

8.1 Upload

Resumable multipart. Client sends a manifest of `{filename, bytes, sha256}` up front; server issues part URLs; `:complete` verifies every checksum and rejects the survey if any mismatch. Enforce `GROMA_MAX_UPLOAD_GB` .

8.2 EXIF and XMP extraction — `exif.py`

Shell out to `exiftool -json -n` . Extract per image:

Field	Source
Make, Model, LensModel	EXIF
FocalLength, FocalLengthIn35mmFilm	EXIF
ImageWidth/Height	EXIF
DateTimeOriginal	EXIF
GPSLatitude/Longitude/Altitude	EXIF GPS
GPSAltitudeRef	EXIF — determines ellipsoidal vs orthometric
RelativeAltitude	XMP <code>drone-dji:RelativeAltitude</code> (AGL)
GimbalPitch/Yaw/RollDegree	XMP <code>drone-dji:</code>

RtkFlag, RtkStdLon/Lat/Hgt	XMP drone-dji: — presence and value 50 means RTK fix
----------------------------	--

Sensor dimensions are not in EXIF. Maintain `fixtures/sensors/sensors.yaml` keyed by `(make, model)` with `sensor_w_mm`, `sensor_h_mm`. Populate for the DJI fleet initially. If unknown, derive from `FocalLengthIn35mmFilm`: $\text{sensor_w_mm} = 36 \times \text{FocalLength} / \text{FocalLength35}$. Flag the survey as `sensor_estimated` in `capture_qa.warnings` — an estimated sensor size propagates directly into GSD and into every px/m figure downstream.

8.3 Quality scoring — `quality.py`

- **Sharpness:** variance of the Laplacian on a greyscale downscale to 1024 px wide. Reject below the 10th percentile of the set *or* an absolute floor, whichever is lower. Percentile-relative because absolute values depend on scene texture.
- **Exposure:** fraction of pixels at 0 or 255. Reject above 5%.
- Record scores on every image whether accepted or not; they go in the QA report.

8.4 Overlap estimation — `overlap.py`

From GPS positions, focal length, sensor size and `RelativeAltitude`, compute each image's ground footprint and estimate:

- **Front overlap:** between consecutive images by timestamp on the same heading.
- **Side overlap:** between nearest images on adjacent flight lines (cluster headings).

Warn below 70% front or 60% side. Block below 60% front.

8.5 Capture classification

Split by `GimbalPitchDegree`: nadir is $-90 \pm 10^\circ$, oblique is -20 to -70° . Add a warning if `oblique_count == 0` — nadir-only capture will not reconstruct masts (`explained.md` §2.10), and mast geometry is the input to camera mounting.

8.6 Video ingest — `video.py`

1. Probe with `ffprobe`: duration, frame rate, resolution, codec.
2. Compute the frame interval from desired overlap: $\text{interval_s} = \text{footprint_along_track_m} \times (1 - \text{overlap}) / \text{ground_speed_m_s}$, with ground speed from GPS track or telemetry.
3. Extract with `ffmpeg -skip_frame nokey` where possible, else at the computed interval.
4. Run the same sharpness and exposure gate.
5. **Surface the rejection rate in the QA report as a blocking item if > 30%.**

Add a permanent warning to `capture_qa.warnings` for any video-sourced survey noting rolling shutter and compression penalties.

8.7 The QA gate

`POST /reconstruct` returns 409 while `capture_qa.blocking` is non-empty. The user must call `qa:acknowledge`, which records who acknowledged what. Blocking conditions:

- Front overlap below 60%
- More than 30% of frames rejected
- Fewer than 20 accepted images
- `georef = 'none'` **and** the site has no scale bar recorded — because the resulting survey cannot support measurement at all

9. Reconstruction adapter

One protocol, three backends.

```
class ReconstructionBackend(Protocol):
    name: str
    version: str
    def available(self) -> bool: ...
    def supports(self, req: ReconRequest) -> bool: ...
    async def run(self, req: ReconRequest,
                  progress: Callable[[str, float, str], Awaitable[None]],
                  ) -> ReconResult: ...
    async def reattach(self, external_id: str, progress) -> ReconResult: ...

@dataclass
class ReconRequest:
    survey_id: str
    image_dir: Path
    gcp_file: Path | None
    srid: int
    quality: Literal["preview", "standard", "high"]
    produce: set[str]          # pointcloud, dsm, dtm, ortho, mesh, poses
    use_3dmesh: bool          # true when oblique images are present

@dataclass
class ReconResult:
    artefacts: dict[str, Path]
    accuracy: AccuracyReport
    poses: dict[str, CameraPose]      # filename -> R, t, intrinsics
    engine: str
```

9.1 fixture.py

Copies pre-baked artefacts from `fixtures/recon/<name>/`, emits plausible staged progress over ~5 seconds, returns a consistent `AccuracyReport` and the exact poses used to render the synthetic survey (§16). Every test above M6 uses this backend.

9.2 odm.py — NodeODM client

Endpoints: `POST /task/new` (multipart: images + options JSON), `GET /task/{uuid}/info` (poll at 5 s), `GET /task/{uuid}/download`, `POST /task/cancel`, `POST /task/remove`.

Options to send:

```
{
```

```

"dsm": True, "dtm": True,
"pc-quality": {"preview": "low", "standard": "medium", "high": "high"}[quality],
"feature-quality": {"preview": "medium", "standard": "high", "high": "ultra"}[quality],
"orthophoto-resolution": 2,          # cm/px
"use-3dmesh": req.use_3dmesh,
"gcp": "gcp_list.txt",              # included in the zip when present
"auto-boundary": True,
"pc-las": True,
}

```

Output path → artefact kind mapping:

Path in the result archive	Artefact kind
odm_georeferencing/odm_georeferenced_model.laz	pointcloud
odm_dem/dsm.tif	dsm
odm_dem/dtm.tif	dtm
odm_orthophoto/odm_orthophoto.tif	ortho
odm_texturing/odm_textured_model_geo.obj (+ mtl, textures)	mesh
opensfm/reconstruction.json	poses
odm_report/stats.json	source of reproj_rmse_px, registered image count

Progress mapping — parse the NodeODM `processingTime` and log stream; map stages to a monotonic 0..1: dataset 0.05 → opensfm 0.35 → openmvs 0.70 → odm_meshing 0.80 → odm_texturing 0.88 → odm_georeferencing 0.92 → odm_dem 0.96 → odm_orthophoto 1.0.

Store the NodeODM task uuid in `job.external_id` immediately after creation. On worker restart, `reattach()` resumes polling instead of re-running a 4-hour job.

Preflight checks before submitting:

- `available()` returns false when `GROMA_NODEODM_URL` is unset or unreachable.
- Disk headroom: require free space $\geq 10 \times$ input size on the NodeODM volume.

9.3 colmap.py — subprocess fallback

Command sequence:

```

colmap feature_extractor --database_path {db} --image_path {images}
      --ImageReader.camera_model OPENCV --ImageReader.single_camera_per_folder 1
colmap sequential_matcher --database_path {db}          # exhaustive_matcher if < 200 images
colmap mapper --database_path {db} --image_path {images} --output_path {sparse}
colmap model_aligner --input_path {sparse}/0 --output_path {sparse}/geo
      --ref_images_path {geo_txt} --ref_is_gps 0 --alignment_type ecef
colmap image_undistorter --image_path {images} --input_path {sparse}/geo
      --output_path {dense}
colmap patch_match_stereo --workspace_path {dense}      # requires CUDA
colmap stereo_fusion --workspace_path {dense} --output_path {dense}/fused.ply

```

- Detect CUDA with `nvidia-smi` ; if absent, raise before `patch_match_stereo` with a clear message rather than hanging. Sparse-only output is still useful (poses for §13), so return a partial `ReconResult` with `pointcloud` absent.
- Substitute `glommap mapper` for `colmap mapper` when the binary is present and the image count exceeds 2,000.
- `model_aligner` needs a text file of `image_name X Y Z` — generate from `source_image.gps`.
- COLMAP produces no DSM, DTM or orthophoto. Generate them in `artefacts.py`: rasterise the fused point cloud with PDAL (`writers.gdal`, `output_type=max` for DSM), run `filters.smrf` for ground classification and `output_type=idw` on ground returns only for DTM. Orthophoto is out of scope for the COLMAP path; mark the artefact absent rather than faking it.

9.4 `artefacts.py`

Read, validate and normalise whatever came back:

- LAZ: verify header SRID, point count, bounds; reproject to the site SRID if needed.
- GeoTIFF: verify SRID, resolution, nodata handling.
- OBJ: verify referenced textures exist.
- Compute sha256 for every artefact; write `artefact` rows.
- **Reject silently-empty outputs.** A 2 KB LAZ or a DSM that is entirely nodata is a failed reconstruction that ODM reported as success. Assert minimum point counts and a maximum nodata fraction (30%).

10. Georeferencing and accuracy reporting

10.1 GCP workflow

1. User creates GCPs with surveyed coordinates and a `control` / `check` role.
2. For each GCP, the API suggests candidate images: those whose GPS position is within the estimated footprint radius of the GCP. Sort by proximity to frame centre.
3. User marks the target in ≥ 3 images per GCP (frontend zoom-to-pixel).
4. `gcp.py` writes `gcp_list.txt`:

```

EPSG:2326
832451.221 816003.874 12.334 2104 1518 DJI_0421.JPG
832451.221 816003.874 12.334 1877 990 DJI_0422.JPG

```

Order: easting, northing, height, pixel-x, pixel-y, filename. First line is the SRID. **Only** `role='control'` GCPs go into the file. Check points are held out.

Refuse to submit with fewer than 3 observations on any control GCP, or fewer than 4 control GCPs total, or all control GCPs within a 20 m height range (vertical scale would be unconstrained).

10.2 RTK path

When $\geq 90\%$ of images have `rtk_fixed = true`, set `georef = 'rtk'` and pass positions through without GCPs.

Still require check points if any are defined.

10.3 Check point residuals

After reconstruction, for each `role='check'` GCP, project its marked observations through the solved camera poses, triangulate, and compare with the surveyed position.

```
check_rmse_h = sqrt(mean(dE² + dN²))
check_rmse_v = sqrt(mean(dH²))
```

These are the only honest accuracy numbers. Control-point residuals are reported separately and labelled as optimistic in the UI and in the PDF.

10.4 Scale check from pitch markings — `scale_check.py`

Independent validation requiring no survey equipment.

- 1. Load the orthophoto. Threshold for white line markings (high luminance, low saturation), morphological cleanup.
- 2. Hough transform → line segments. Cluster into two perpendicular families; find the largest enclosed rectangle.
- 3. Compare its dimensions against a library of regulation dimensions:

```
football_full:    {length: 105.0,  width: 68.0,  tol: 0.0}
football_range:  {length: [90,120], width: [45,90]}      # ambiguous, warn only
basketball_fiba: {length: 28.0,   width: 15.0}
tennis_doubles:  {length: 23.77,  width: 10.97}
netball:         {length: 30.5,   width: 15.25}
handball:        {length: 40.0,   width: 20.0}
```

- 4. Report `scale_error_pct = (measured/nominal - 1) × 100`.
- 5. Manual fallback: the user clicks four corners in the orthophoto and picks the court type. Always offer this; automatic detection will fail on worn markings.

Warn above 0.5% error; block survey completion above 2% — that is a georeferencing failure, not a measurement nuance.

11. Artefact processing and tiling

`packages/tiles/` . Runs as a job after reconstruction succeeds.

Input	Process	Output artefact
LAZ point cloud	Rebase to site origin, then <code>py3dtiles</code> convert (or Entwine EPT → 3D Tiles)	<code>tiles_pc</code>
Textured OBJ	Rebase, decimate to a screen-space-error budget, Draco-compress → glTF	<code>tiles_mesh</code>
DTM GeoTIFF	Rebase, resample to a regular local-ENU grid, store as <code>.npz</code>	<code>terrain_grid</code> — consumed directly by the coverage kernel
DSM		

GeoTIFF	Rebase, resample	used by extraction for above-ground residual
Orthophoto	Convert to COG, generate XYZ tiles	<code>tiles_ortho</code> — ground texture in the viewer

Rebasing happens at tile generation time, once. The tiles and the terrain grid are in local ENU. Nothing downstream sees projected coordinates. See `explained.md §3.3` for why.

Terrain grid resolution: 0.5 m by default, configurable. It is both the coverage kernel's occluder and the source of per-cell evaluation height.

12. Structure extraction

`packages/segment/` . Input: LAZ point cloud + DTM. Output: `structure` rows in `pending` , plus `mount_point` rows.

12.1 Pipeline

```
def separate_ground(points, cloth_resolution=0.5, rigidity=3, threshold=0.15)
    -> tuple[ground, above_ground]
    """Cloth simulation filter. Use PDAL filters.csf or the CSF pip package.
    Cross-check against the DTM: the CSF ground surface and the DTM should agree
    to within 10 cm; a larger discrepancy is a reconstruction problem."""

def cluster(points, voxel=0.05, eps=0.35, min_pts=12) -> list[np.ndarray]
    """Voxel downsample then DBSCAN (Open3D). Seed the RNG."""

def descriptors(cluster_points) -> ClusterDescriptors
    """From the eigenvalues  $\lambda_1 \geq \lambda_2 \geq \lambda_3$  of the position covariance:
        linearity    =  $(\lambda_1 - \lambda_2) / \lambda_1$ 
        planarity    =  $(\lambda_2 - \lambda_3) / \lambda_1$ 
        sphericity   =  $\lambda_3 / \lambda_1$ 
        verticality  =  $|\mathbf{e}_3 \cdot (0,1,0)|$  where  $\mathbf{e}_3$  is the smallest-eigenvalue vector
    Plus: height above local ground, plan-view extent and radius,
        point count, point density."""

def fit_cylinder(points, seed) -> tuple[CylinderPrim, float] | None    # RANSAC, +rmse
def fit_vertical_plane(points, seed) -> tuple[ExtrudedPolyline, float] | None
def fit_box(points) -> tuple[BoxPrim, float]                          # oriented bbox

def classify(desc, fits) -> tuple[StructureClass, float]
def extract_mounts(structure) -> list[MountPoint]
```

12.2 Classification rules — start here, tune per site type

Keep all thresholds in one `ExtractionConfig` dataclass, serialised with the job so a result can be reproduced.

Class	Rule
pole	<code>verticality > 0.95 and plan_radius < 0.4 and height > 4 and cylinder_inlier_ratio > 0.7</code>
fence	<code>planarity > 0.85 and thickness < 0.4 and plan_length > 3 and height < 4</code>

stand	volume > 100 m ³ and dominant horizontal plane above 2 m and adjacent to the playing surface
building	volume > 20 m ³ and dominant horizontal plane above 2 m
goal	2 < height < 3.5 , rectangular plan outline, point density below the 20th percentile (open frame)
vegetation	sphericity > 0.25 and no cylinder or plane fit above 0.5 inlier ratio
other	everything else, confidence 0.3, always requires review

Rule-based first, not learned. The descriptors separate these classes well on open sports grounds, the rules are inspectable in review, and there is no training data until the system has been in production. The accept/reject rate from review is the metric that tells you when a learned classifier is worth building.

12.3 Post-processing

- **Merge collinear fence clusters** into single polyline runs, so a perimeter fence is 4 structures rather than 40.
- **Set porosity**: default 0.85 for `fence` classified as mesh (thin, low point density), 0 for solid walls. Expose in review.
- **Set mountable**: true for `pole`, `building`, `stand`.
- **Extract mount points**: pole → 1 m below the head, plus 3 m and 6 m options; building/stand → each roof corner and each façade midpoint at parapet height.
- **Evidence**: for each structure, find the 3 source images whose pose best views it (highest projected area, smallest incidence angle); store `{image_id, bbox}` for the review UI.

12.4 Review

`PATCH /structures/{id}` accepts state, class and primitive changes. **Class change triggers** `refit` with the new class's fitter — a cluster called `fence` and one called `building` get different thicknesses and cast different shadows. Manual primitive edits set `origin = 'adjusted'`.

`POST /surveys/{id}/structures` creates a manual structure for anything extraction missed entirely (`origin = 'manual'`). Required: reconstruction never recovers everything.

12.5 Validation

Run extraction on the synthetic survey (§16) and assert it recovers the authored `site_alpha` primitives:

- Every authored structure has a matching extracted structure of the correct class.
- Pole positions within 10 cm, radii within 5 cm, heights within 20 cm.
- Fence lines within 15 cm, heights within 15 cm.
- Building box corners within 25 cm.
- No more than 3 spurious extra candidates above confidence 0.5.

This is `tests/integration/test_extraction_recovers_truth.py` and it is the acceptance criterion for M9.

13. Semantic lift from source imagery

`packages/segment/lift.py` . Improves classification where geometry is ambiguous — floodlight mast vs flagpole vs CCTV pole vs lightning conductor.

1. Run 2D instance segmentation over accepted source images. Start with a fine-tuned `yolo11-seg` on the project's classes; SAM 2 plus a crop classifier is the alternative when labelled data is thin.
2. For each 3D point and each image whose pose saw it:
 - Project through the pose and intrinsics (including distortion) to a pixel.
 - **Visibility check:** compare the point's camera-frame depth with the dense depth map at that pixel. Skip if occluded (tolerance 0.5 m).
 - Read the mask label; accumulate one vote.
3. Point label = argmax of votes. Cluster label = majority of its points.
4. Combine with the geometric classifier: geometry decides the primitive, semantics decides the class. Where they disagree, lower confidence and force review.

Requires the `poses` artefact and depth maps. Depth maps are large; regenerate on demand from the mesh rather than storing them.

GPU-only in practice. The system is complete and correct without this stage — it raises the accept rate in review, it does not gate anything.

14. Measurement and uncertainty

14.1 Snapping

Priority order, first hit wins:

1. **Fitted primitive feature** — pole axis, pole top, fence line, box edge/corner. $\sigma_{\text{snap}} = \text{fit_rmse} / \sqrt{\text{point_count}}$.
2. **Terrain surface** (DTM). $\sigma_{\text{snap}} = \text{terrain grid noise estimate}$.
3. **Local plane** — fit a plane to points within 0.5 m of the ray hit. $\sigma_{\text{snap}} = \text{plane residual RMS}$.
4. **Nearest raw point.** $\sigma_{\text{snap}} = \text{local point spacing}$.

Record `snap_mode` on every measurement.

14.2 Types

3D distance, horizontal distance, vertical difference, height above local ground, polyline length, planar area, footprint area, volume to a reference plane, slope, clearance between two objects.

14.3 Uncertainty propagation

Per snapped point:

$$\begin{aligned}\sigma_h &= \sqrt{(k_h \cdot \text{GSD})^2 + \text{check_rmse}_h^2 + \sigma_{\text{snap}}^2} & k_h &= 1.5 \\ \sigma_v &= \sqrt{(k_v \cdot \text{GSD})^2 + \text{check_rmse}_v^2 + \sigma_{\text{snap}}^2} & k_v &= 3.0\end{aligned}$$

Where check points are absent, substitute control-point RMSE and add a 50% penalty; where neither exists, the

survey is `georef='none'` and measurement is refused.

For a distance between A and B, project each point's anisotropic uncertainty onto the measurement direction and combine in quadrature.

Formatting: round the value to the decade of its uncertainty and always render the tolerance. `47.82 m ± 0.03`, never `47.8213 m`. Apply this in a single `format_measurement()` helper used by the API, the frontend and the PDF export.

15. Placement optimisation and reporting

15.1 Optimisation — `packages/coverage/optimise.py`

1. **Candidates** from `mount_point` rows on accepted structures, crossed with a discretised option set: pan at 15° steps, tilt at {10, 15, 20, 25, 30, 40}°, lens from a configured catalogue of real camera models.
2. For each candidate, compute coverage once and store per-tier bitsets over the grid. This is the expensive step; parallelise across workers and cache by `(scenario_id, candidate_hash, kernel_version)`.
3. **Greedy maximum coverage:** repeatedly select the candidate adding the most new weighted area at or above the target tier. Guarantee: $\geq 1 - 1/e \approx 63\%$ of optimal.
4. Constraints: maximum camera count, maximum cable run from a set of head-end positions, per-mount-point load and count limits, minimum and maximum mount height.
5. Report each selection as **"adds X m² that nothing else covers"** — the greedy marginal gain. That number is the justification, and it is why greedy is preferred over an ILP whose output cannot be explained.

15.2 PDF report

Contents, in order:

1. Site and survey identification, flight date, engine and version.
2. **Accuracy statement** — GSD, reprojection RMSE, check-point RMSE, scale error, georeferencing method. If `georef='none'`, a full-page notice that no dimension in the report is metrically valid.
3. Structure schedule: every accepted structure, class, dimensions, reviewer, date. Rejected structures listed separately with reason.
4. Camera schedule: position, mount, pan/tilt, lens, sensor, resolution, and per camera the DORI ranges and unique coverage area.
5. Coverage plan per DORI tier, over the orthophoto.
6. Blind-spot plan with polygons and areas.
7. Redundancy plan (cells by camera count).
8. Scenario comparison where applicable: the delta table and the newly-blind area.
9. `kernel_version`, `coverage_run.id`, computation parameters, timestamp.

Every number in the PDF comes from the persisted `coverage_run`, never recomputed at render time.

16. Synthetic ground truth

Two scripts that make the entire photogrammetry pipeline testable without a drone.

16.1 `scripts/synthesise_site.py`

Input: `fixtures/sites/site_alpha.json` (the authored proxy model). Output into `fixtures/recon/site_alpha/`:

- `pointcloud.laz` — points sampled on every primitive surface plus a textured ground plane, at a configurable density (default 400 pts/m²), with Gaussian noise (default $\sigma = 1.5$ cm) and a configurable outlier fraction (default 0.5%).
- `dsm.tif`, `dtm.tif` — rasterised at 0.5 m.
- `ortho.tif` — the ground plane with pitch markings drawn at regulation dimensions. This gives the scale check (§10.4) a known-correct input.
- `truth.json` — the authored primitives, for the extraction assertion in §12.5.

Include realistic defects, controlled by flags, so extraction is tested against what it will actually meet: `--thin-poles` (drop 40% of points above 8 m on cylinders), `--fence-gaps` (remove two 3 m sections), `--floating-artefacts` (add 5 noise blobs).

16.2 `scripts/render_survey.py`

Renders `site_alpha` from a simulated flight — nadir cross-hatch plus an oblique perimeter orbit — using an offscreen renderer.

Output: `fixtures/recon/site_alpha/images/` plus `poses_truth.json` containing the exact extrinsics and intrinsics used.

This makes reconstruction itself testable: run ODM or COLMAP on the rendered images and compare recovered poses against `poses_truth.json`.

Acceptance for M6: on the synthetic render set, $\geq 95\%$ of images register, and recovered camera positions agree with truth to within $3 \times$ the rendered GSD after similarity alignment.

17. Milestones

Each has a completion criterion. Do not start the next until the previous one's tests pass.

	Milestone	Contents	Done when
M0	Skeleton	Repo layout, uv workspace, docker-compose, Makefile, ruff/mypy/pytest, CI, <code>packages/contracts</code> complete (§4), TS type generation	make test and make lint clean; contracts importable from every package; T14 passes
M1	Coverage kernel	<code>geo/optics.py</code> , <code>coverage/*</code> including terrain and <code>reference.py</code>	T1–T14 pass; 173k cells $\times$ 6 cameras $\times$ terrain under 800 ms; golden fixture within ± 0.3 pp
M2	Synthetic ground truth	<code>scripts/synthesise_site.py</code> , <code>scripts/render_survey.py</code> , fixtures committed or generated in CI	Fixture cloud loads; <code>truth.json</code> matches <code>site_alpha.json</code> ; ortho scale check returns $< 0.1\%$ error

M3	Frontend, offline	Port the prototype to React/TS; TS kernel; scene, panels, heatmap, tents, camera editing	Kernel parity test passes (< 0.5% cells differ by > 1 px/m); prototype functionality reproduced
M4	API and database	Migrations, SQLAlchemy models, sites/scenarios/cameras/tents/coverage/measurement endpoints, auth roles, seed	Frontend loads from API; integration tests cover every endpoint; <code>groma seed --reset</code> produces a working demo
M5	Jobs and reporting	arq + Redis, WebSocket progress, persisted coverage runs with kernel version, scenario clone and compare, PDF export (§15.2)	0.25 m coverage run queues, streams, persists, exports a PDF whose numbers match the API
M6	Capture ingest	Upload, EXIF/XMP, sensor table, quality scoring, overlap estimation, video keyframes, QA gate (§8)	QA report generated for the synthetic render set and for a real DJI folder; blocking gate enforced; video path reports rejection rate
M7	Reconstruction	<code>ReconstructionBackend</code> , fixture + ODM + COLMAP backends, re-attach, artefact validation (§9)	Fixture recon runs end to end through jobs; ODM produces the full artefact set from the rendered images; poses within 3× GSD of truth
M8	Georeferencing	GCP CRUD and marking UI, <code>gcp_list.txt</code> , RTK path, check-point residuals, pitch-marking scale check (§10)	Check-point RMSE reported; scale check passes on the synthetic ortho; <code>georef='none'</code> blocks measurement with 409
M9	Tiling	Point cloud → 3D Tiles, mesh → glTF, DTM → terrain grid, ortho → XYZ; local-origin rebasing (§11)	Viewer streams the synthetic cloud; coverage kernel consumes the terrain grid; T10/T11 pass against real terrain
M10	Structure extraction	CSF, clustering, descriptors, RANSAC fitting, classification, merging, mount points, evidence crops (§12)	<code>test_extraction_recovers_truth.py</code> passes at the tolerances in §12.5
M11	Review UI	Structure list, evidence thumbnails, accept/reject/reclassify/adjust, refit on class change, manual structures, typed rejection	Reviewing the extracted synthetic survey reproduces <code>site_alpha</code> coverage stats within 1 pp of the authored model
M12	Measurement	Snapping hierarchy, all measurement types, uncertainty propagation, <code>format_measurement()</code> (§14)	Measured pitch on the synthetic ortho is 105.0 ± tolerance; no code path formats a value without its tolerance
M13	Semantic lift	2D segmentation, back-projection voting with visibility check (§13)	Pole subclasses correctly separated on a labelled set; degrades gracefully when unavailable
M14	Optimisation	Candidate generation, bitsets, greedy selection, constraints (§15.1)	On <code>site_alpha</code> , greedy with 6 cameras beats the hand-placed 4-camera baseline on Recognise area; marginal gains reported
M15	Deployment	AutoDL bootstrap, nginx, supervisord, runbook (§19)	Fresh box to working demo in one command; runbook entries written and each one tested by causing the failure

18. Test strategy

The dangerous tests here are the ones that pass by testing nothing. A coverage assertion of “right shape, some non-zero values” passes for nearly every bug this system can have. **Every test must have an expected value derived independently of the code under test** — from arithmetic, from `reference.py`, or from a committed golden file.

Tier	Tool	Scope
Unit	pytest	Optics, kernel analytics (§6.6), fitting, CRS, EXIF parsing, GCP file generation
Property	hypothesis	Monotonicity, foreshortening bound, basis orthonormality under random pan/tilt/roll, rebasing round-trip
Reference parity	pytest	Fast kernel vs <code>reference.py</code>
Golden	pytest	<code>site_alpha</code> stats within ± 0.3 pp
Recovery	pytest	Extraction recovers authored truth (§12.5); reconstruction recovers rendered poses (§16.2)
Integration	pytest + testcontainers	API against real PostGIS; job lifecycle including worker restart and re-attach
Cross-language	Playwright	TS kernel vs Python kernel
E2E	Playwright	Upload → QA → reconstruct (fixture) → extract → review → place camera → coverage → PDF

Unit suite target: **under 5 seconds**. Push anything slower into integration.

Seed all randomness explicitly — RANSAC, DBSCAN initialisation, synthetic noise — and pass the seed through the job record so any result can be reproduced.

Two tests that pay for themselves repeatedly: T14 (import graph) and T8 (self-occlusion).

19. Deployment

19.1 Services

```
nginx      :6006 → /      static SPA
              /api    → uvicorn :8000
              /ws     → uvicorn :8000 (upgrade)
              /tiles  → static, from the artefact store

uvicorn    :8000 FastAPI
arg worker  background jobs
postgres   :5432 localhost only
redis      :6379 localhost only
nodeodm    :3000 where Docker is available; otherwise unset GROMA_NODEODM_URL
```

Single-port ingress because AutoDL exposes one custom-service port.

19.2 `deploy/autodl/bootstrap.sh`

- Probe the Postgres binary path across versions 14/15/16; install matching PostGIS (`postgresql-<v>-postgis-3`) with the same probe.
- Install system dependencies: `exiftool` , `ffmpeg` , `gdal-bin` , `pdal` , `libgl1` .
- If Node is absent, warn and serve a pre-built `dist/` if one was shipped; do not fail the bootstrap.
- Create the venv; `pip install` with `--break-system-packages` where required.
- `alembic upgrade head` .
- Render supervisord and nginx configs from templates with resolved paths.
- **Detect Docker** (`docker info`). If unavailable, leave `GROMA_NODEODM_URL` unset and print that only the fixture, COLMAP and import reconstruction paths are available on this host.
- Detect CUDA (`nvidia-smi`); record GPU memory in the runbook output.
- Start supervisord; print the service URL and seeded credentials.

Capture and record on first deploy: `nvidia-smi` , `df -h` . GPU memory bounds the feasible `--pc-quality` ; disk bounds artefact retention. Both belong in the runbook as measured values.

19.3 Configuration

Validated by a `Settings` model at startup so a missing variable fails immediately.

```
GROMA_DATABASE_URL
GROMA_REDIS_URL
GROMA_ARTEFACT_ROOT
GROMA_NODEODM_URL      # empty → ODM backend reports available() = False
GROMA_COLMAP_BIN        # empty → COLMAP backend unavailable
GROMA_DEFAULT_SRID
GROMA_JWT_SECRET
GROMA_MAX_UPLOAD_GB
GROMA_KERNEL_MAX_CELLS # guard against a 0.05 m grid request
```

19.4 Runbook entries

Each is symptom → diagnosis → fix. Write them, then cause each failure to check them.

- Reconstruction job stuck: distinguishing hung from slow; where NodeODM logs live
- Disk full mid-reconstruction (ODM intermediates run 5–10× input size)
- Worker restarted during a 4-hour reconstruction: re-attach via `job.external_id`
- Coverage run times out: grid too fine, or occluder count exploded from bad extraction
- Frontend numbers disagree with the PDF: kernel parity has drifted
- Reconstruction "succeeded" with an empty point cloud
- Postgres connection pool exhausted

- Survey needs re-running: supersede, never mutate
- GCP marking rejected: fewer than 3 observations, or insufficient height spread

19.5 Security

- Database and Redis bind to localhost, behind nginx.
 - Structure review and GCP marking require an authenticated `surveyor`.
 - Uploaded imagery is user content: validate type and size, never execute.
 - Rotate any credential that has been pasted into a chat.
-

20. Session guidance

- **One milestone per session.** Break it into sub-tasks and do one per prompt. "Implement the coverage kernel" is too large; "implement `occluders.py` with the slab and cylinder tests, plus T1–T3" is right.
- **Quote the spec section into the prompt.** §4, §5, §6, §8, §9 and §12 are written to be pasted directly.
- **Write tests alongside implementation, with the expected values from this document in the prompt.**
- **`make test` at the start and end of every session.**
- **Change `packages/coverage` and `apps/web/src/kernel` in the same session** so parity is maintained in one pass.
- **When a coverage map looks wrong, reach for `reference.py` first.** Run both, diff the arrays, find the first differing cell, work backwards. Faster than reasoning about vectorised NumPy.
- **When extraction looks wrong, run it against the synthetic truth,** not against real data. You know the answer there.

The failures most likely to cost a day

1. A pan/tilt sign error producing coverage in the wrong quadrant that looks superficially fine. T9.
2. A broadcasting bug in the vectorised kernel producing a map that is smooth, plausible and wrong. T5.
3. Self-occlusion by the mount structure. T8.
4. A survey reconstructed without scale, measured confidently, and only noticed when the numbers reach a site plan. §10.4 and the 409 gate.
5. Mixed height datums putting cameras tens of metres underground. Every stored height is labelled.